

It's **not just a “mode of communication between client and server”** in the traditional sense like HTTP or REST. Instead:

- MCP is a **standardized protocol** that defines how **AI models (like LLMs)** and **external tools/services** talk to each other.
- It acts like a **common language or contract** between the AI and any tool (databases, APIs, search engines, compliance systems, etc.).

❓ When a service changes endpoints, your integration breaks. Developers scramble to

As you add more tools (KYC, AML, credit bureaus, payment rails), the number of unique integrations grows, increasing maintenance and risk.

❓ Rolling out new capabilities requires coordination across teams and vendors; even small changes can ripple through systems.

How MCP changes the game

MCP (Model Context Protocol) helps AI systems to communicate with external tools like some other particular data sources like Wikipedia. Instead of every client integrating each tool's unique REST API, tools expose a consistent interface via MCP. The benefits:

How MCP changes the game

MCP (Model Context Protocol) standardizes how AI systems communicate with tools. Instead of every client integrating each tool's unique REST API, tools expose a consistent interface via MCP. The benefits:

- **Plug-and-play tools:** Service providers ensure MCP compliance; clients connect once and swap tools without rewriting code.

Think of MCP as the “USB standard” for AI-tool communication—consistent ports, interchangeable devices.

Banking example: End-to-end loan origination with agentic AI and MCP

Scenario overview

A mid-sized bank wants to automate consumer loan origination while staying compliant. They use an agentic AI system with multiple agents, each calling MCP-compliant tools:

- **Intake agent:** Conversational onboarding, document collection, and consent.
- **Identity agent:** KYC/AML checks, PEP/sanctions screening.

- **Credit agent:** Credit bureau pulls, income verification, debt-to-income calculation.
- **Risk agent:** Model scoring, fraud signals, policy rules.
- **Compliance agent:** Regulatory checks, adverse action logic, audit trail.
- **Ops agent:** Decision communication, offer generation, e-sign, and funding coordination.

Workflow with MCP

1. Customer intake

- **Intake agent** chats with the applicant, explains terms, and collects documents.
- Calls MCP tool: `document_ingest` to store and index PDFs/images.
- Calls MCP tool: `consent_manager` to record explicit consent with standardized metadata.

2. Identity verification

- **Identity agent** calls MCP tool: `kyc_verify` with applicant data.
- If needed, calls `sanctions_screen` and `pep_check`.
- Tools return structured results (status, risk flags, evidence links) in a consistent MCP format.

3. Credit assessment

- **Credit agent** calls MCP tool: `credit_bureau_pull` (Equifax/Experian-like) via MCP.
- Calls `income_verify` (e.g., payroll APIs or bank statement analysis).
- Computes debt-to-income using standardized outputs—no custom parsing per provider.

4. Risk scoring and fraud checks

- **Risk agent** calls MCP tool: `risk_model_score` with features from prior steps.

Traditional AI Workflow (Without MCP)

Credit Assessment

- Credit agent calls REST APIs of credit bureaus (Equifax, Experian, TransUnion).

- Each bureau has different request/response formats.
- If one bureau updates its API, developers must patch code immediately or risk breaking the workflow.

! Problems in Traditional Workflow

- **High maintenance cost:** Every API update requires developer intervention.
- **Fragile integrations:** If one service changes, the workflow can break.
- **Vendor lock-in:** Switching providers means rewriting large parts of code.
- **Slow innovation:** Adding new tools takes weeks/months due to integration complexity.
- **Inconsistent data formats:** Developers must manually normalize outputs across services.

✓ MCP-Enabled Workflow (With MCP)

- **Plug-and-play:** Developers connect once; if the bank switches from Equifax to Experian, no code changes are needed — both comply with MCP.
- **Stable contracts:** Even if providers update their internal APIs, the MCP interface remains consistent.

🏦 Banking Example

Imagine a bank's AI assistant needs to check a customer's credit score:

- **Without MCP (traditional REST):**
 - Developer writes integration for Equifax's REST API.
 - If the bank switches to Experian, the developer must rewrite code for Experian's API.
 - If Equifax changes its API version, the integration breaks until patched.
- **With MCP:**
 - The AI calls `credit_bureau_pull` via MCP.
 - Whether the provider is Equifax, Experian, or TransUnion doesn't matter — all expose the same MCP capability.
 - If Experian updates its backend API, the MCP layer absorbs the change. The bank's AI workflow keeps running without developer intervention.

User → MCP Host → MCP Client → MCP Server → Tools → Context → LLM → Output.

It sounds like you are following a tutorial on the **Model Context Protocol (MCP)** and specifically how to use **Claude Desktop** as a host.

Here is a concise summary of the key concepts and steps covered in the video:

1. The Core Components

The video explains the relationship between the three main pillars of MCP:

- **MCP Host:** The application you use (in this case, **Claude Desktop**). It contains the **MCP Client**.
- **MCP Client:** The internal mechanism that uses the protocol to talk to servers.
- **MCP Server:** A service (like Airbnb, Google Search, or a local database) that provides specific "tools" or data to the AI.

2. Setting Up the Host (Claude Desktop)

- The video guides you through downloading and installing the **Claude Desktop** app (available for Windows and Mac).
- It highlights that this app acts as the user interface where the AI (like Claude 3.5 Sonnet) can interact with external tools once they are configured.

3. The Configuration Process

The "magic" happens in a specific configuration file.

- **Location:** In Claude Desktop, you go to Settings -> Developer -> Edit Config.
- **File Name:** `claude_desktop_config.json`.
- **Method:** You add JSON blocks to the `mcpServers` section. These blocks tell the host which command to run (usually via `npx` or `node`) to start a specific server.

4. Practical Examples of Integration

The video demonstrates how to connect two real-world services:

- **Airbnb:** By adding the Airbnb MCP server config, Claude gains the ability to search for real-time property listings and details.
- **DuckDuckGo:** By adding this server, Claude can perform live web searches to get current news (like AI updates from 2025/2026) that aren't in its original training data.

5. Critical Troubleshooting Tip

A key takeaway from the video is that **Claude Desktop must be forcefully restarted** (via Task Manager or Activity Monitor) after editing the JSON config file. Simply closing the window isn't enough to refresh the new server integrations.

Key Takeaway

MCP allows you to turn a standard AI chatbot into a powerful workstation that can browse the web, search travel sites, or query databases simply by "plugging in" third-party servers through a standardized JSON configuration.